

Sistemas Operacionais

II - Estruturas do S.O

Professor: Eduardo Duarte

Agenda

01

Características e Serviços dos S.Os

02

Interface Usuário / S.O

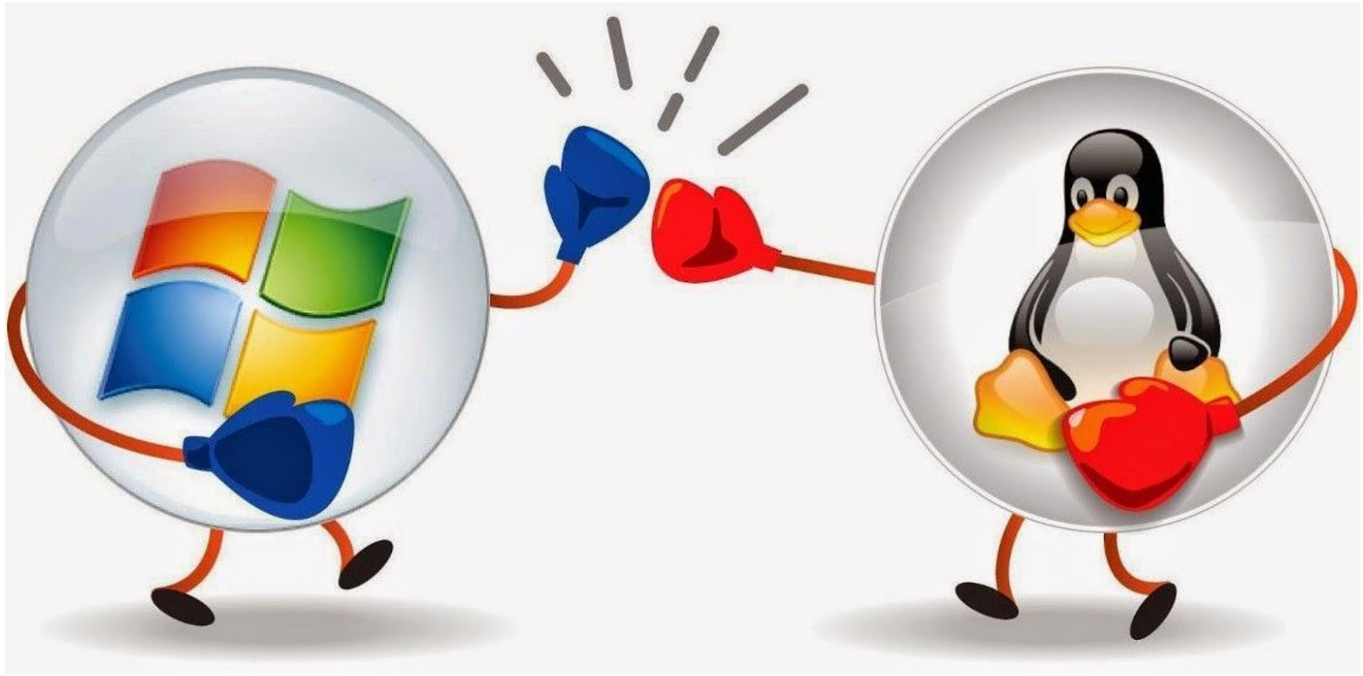
Parte I

Características e Serviços dos S.Os

Interface Usuário / S.O



Funcionamento aplicações de alguns S.Os



Tanto o Windows quanto o Linux são sistemas operacionais com suas próprias vantagens e que diferem em funcionalidade e facilidade de uso.



Algumas características iniciais

Linux

- Distribuição gratuita
- Também há distribuições pagas
- Kernel Linux
- Desenvolvido pela comunidade sob supervisão de Linus Torvalds
- Compatível com muitos hardwares

Windows

- Distribuições pagas
- Desenvolvido pela Microsoft
- Kernel Windows NT
- Interface gráfica como ponto forte
- Compatível com muitos hardwares



Algumas características iniciais

Windows

- Versão para servidor e desktop
- Surgiu como complemento do MS-DOS
- Foco em constante investimento em interface gráfica





Algumas características iniciais

Linux

- Linux é um Kernel
- Ubuntu, Fedora, Debian, Kali...
- Foco em trabalhar com linhas de comando, apesar de possuir interface gráfica na maioria das distribuições





Algumas características iniciais

Linux

- Os usuários podem alterar seu Kernel para fazer alterações no S.O.
- Alterar preferências de usuário, modo de operação, ou até mesmo desenvolver uma distribuição nova.

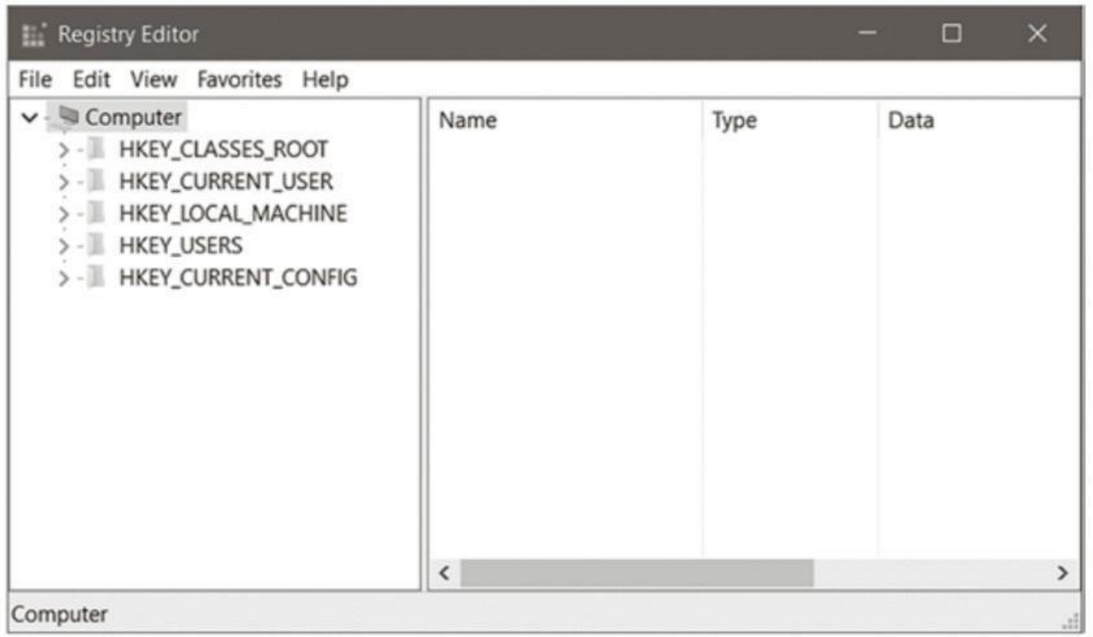




Principais diferenças entre Windows e Linux

Armazenamento de opções de configurações

O Windows usa um único banco de dados de opções de configuração com a ajuda de seu registro.



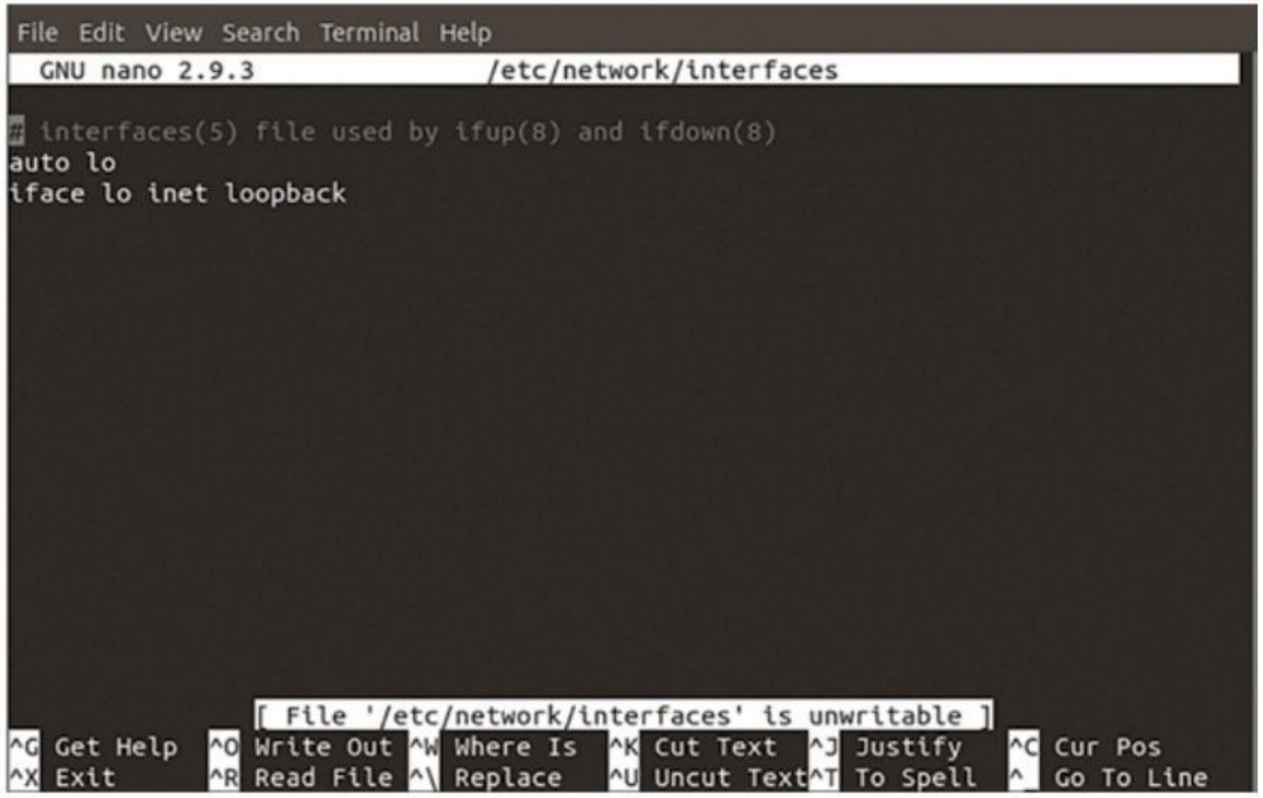


Principais diferenças entre Windows e Linux

Armazenamento de opções de configurações

O Linux não fornece esse recurso; não há registro e os arquivos de **configuração de todo o sistema estão no diretório raiz**.

Arquivos de configuração **específicos do usuário**, geralmente, estão localizados em diretórios ocultos na pasta inicial.



Exemplos de Configuração: Windows vs Linux

Windows Registry (Registro)

- Base de dados hierárquica e centralizada para todo o sistema e usuários.
- Organizado em chaves principais (ex: **HKEY_LOCAL_MACHINE**, **HKEY_CURRENT_USER**).
- A edição direta exige ferramentas específicas como o **Regedit**.
- Uma corrupção no registro pode impedir a inicialização completa do sistema operacional.

Linux Plain Text (Arquivos de Texto)

- Configurações globais residem em arquivos legíveis dentro de **/etc/**.
- Configurações de usuário ficam ocultas na pasta pessoal (**~/.config/**).

Terminal - /etc/network/interfaces

```
# Configuração de interface de rede local no Linux
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
address 192.168.1.100
netmask 255.255.255.0
gateway 192.168.1.1
```

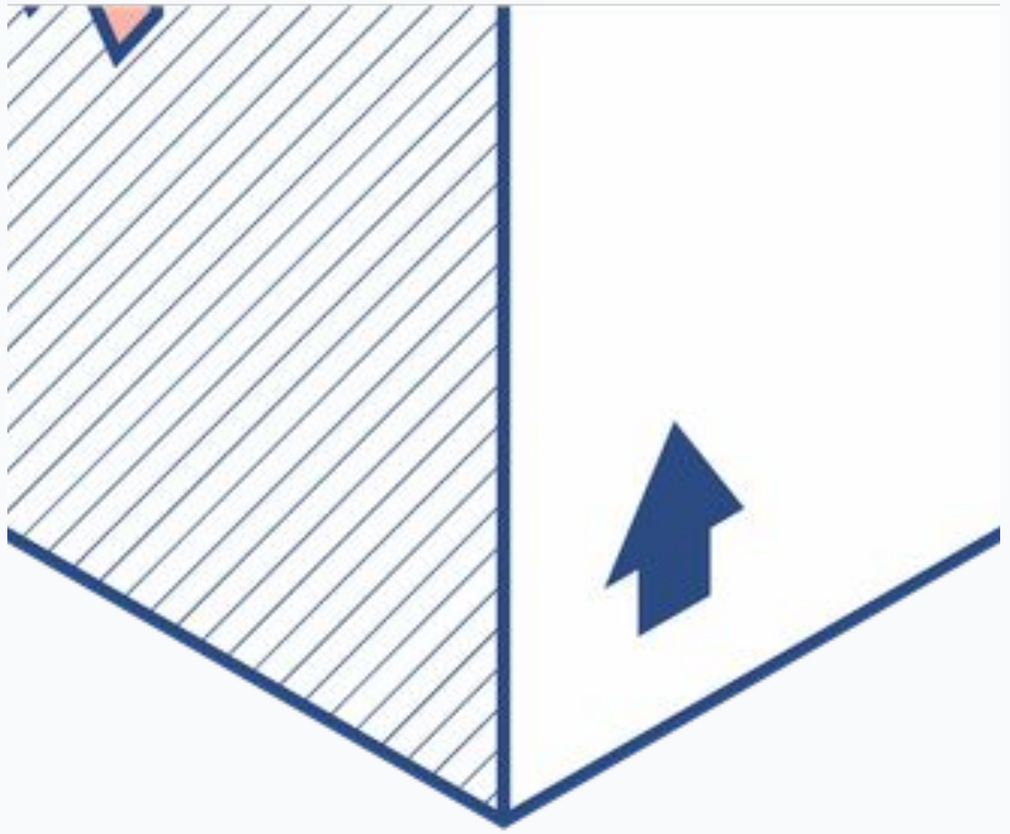


Principais diferenças entre Windows e Linux

Armazenamento de opções de configurações

Nesse sentido, é melhor que o Windows, pois não há chance de falha na configuração do sistema.

Se um arquivo de configuração for danificado, apenas essa função será interrompida, e o restante funcionará.



Principais diferenças: Linux

01

Backup de Configurações

O Linux facilita o backup de arquivos de configuração — é o mesmo que copiar todos os outros arquivos — e a solução de problemas.

02

Sistema de Raiz Única

Os sistemas de arquivos não recebem letras, como no Windows. Existe um único diretório raiz estruturado sob o caminho único **"/"**.

03

Montagem de Unidades

O Linux monta novas unidades e dispositivos diretamente em pastas dentro do próprio sistema de arquivos raiz de maneira transparente.



Principais diferenças entre Windows e Linux

WINDOWS

01

Busca e Instalação Manual

Windows, precisamos buscar programas e instaladores se tivermos que encontrar um programa para executar uma tarefa e instalá-lo usando seu próprio instalador.

02

Falta de Garantia e Segurança

Não há garantia de que ele seja instalado corretamente e livre de vírus.

Principais diferenças entre Windows e Linux

WINDOWS

Instalação de Programas

No Windows, o processo clássico de buscar instaladores na internet frequentemente expõe o sistema a softwares adicionais embutidos e indesejados (como adwares e barras de ferramentas).





Principais diferenças entre Windows e Linux

LINUX

01

Repositórios de Software

O Linux evita o estilo de busca e download de executáveis soltos na internet, introduzindo o conceito seguro de repositórios centrais de programas.

02

Instalação Simplificada

Quando queremos instalar um programa, simplesmente usamos o utilitário do sistema para buscar, escolher e realizar a instalação com apenas um clique.

03

Gestão de Dependências

O gerenciador de pacotes resolve e instala automaticamente todos os requisitos adicionais necessários, tanto na instalação quanto na remoção do software.



Principais diferenças entre Windows e Linux

WINDOWS

01

Controle de Acesso

Quanto ao controle de acesso, os usuários do Windows costumam fazer login diretamente como administradores do sistema.

02

Vulnerabilidade

Como consequência dos privilégios elevados, o sistema torna-se altamente suscetível a infecções por vírus e malware.

03

Ameaças Invisíveis

Muitos programas nocivos são executados em segundo plano pelos próprios usuários, sem que eles percebam a atividade.



Principais diferenças entre Windows e L

LINUX

01

Administrador Root

No Linux, o administrador é chamado de “root” e só se deve usar o acesso root quando realmente necessário.

02

Avisos de Senha

Interfaces modernas avisarão o usuário sobre a senha de root de forma segura e transparente quando for preciso.

03

Uso do Comando Sudo

Sempre devemos fazer o login como usuário comum. Em caso de necessidade específica, utilizamos o comando "sudo" para permissões de admin.



Principais diferenças entre Windows e Linux

LIBERDADES DE ACESSO

01

Linux

No Linux, o usuário tem acesso ao código-fonte do kernel e o altera de acordo com a sua necessidade. Isso tem suas próprias vantagens, como erros no sistema operacional, que serão corrigidos em um ritmo rápido, e desvantagens, como os desenvolvedores poderem tirar proveito de qualquer fraqueza no sistema operacional.

02

Windows

No Windows, o usuário não possui acesso ao código-fonte – somente os membros do grupo selecionado possuem esse acesso.



Principais diferenças entre Windows e Linux

01

Linux

Variações: Altamente personalizável com diversas distribuições criadas para as necessidades do usuário.

Licenciamento: Sob a licença GPL, garantindo a liberdade de modificar, reutilizar e vender o sistema.

02

Windows

Variações: Poucas opções de personalização nativas, limitadas principalmente a cores e formatos de janelas.

Licenciamento: Proprietário e restrito pela Microsoft, sem acesso ao código-fonte e limitado por número de máquinas.

VARIAÇÕES E LICENCIAMENTO



Principais diferenças entre Windows e Linux

01

Linux

Linha de comando: No Linux, a linha de comando é uma ferramenta muito útil para tarefas administrativas e diárias, mas, para usuários finais, não faz muita diferença.

Nível de execução: O Linux tem a capacidade inerente de parar em diferentes níveis de execução; com isso, podemos trabalhar usando CLI e GUI se houver problemas.

02

Windows

Linha de comando: No Windows, temos a linha de comando tradicional e o PowerShell, que possui algumas ferramentas encontradas no Linux também.

Nível de execução: Em caso de problemas, precisamos reiniciar no nível de execução 3 como administrador/root para localizar e corrigir o problema.

COMANDO E NÍVEIS DE EXECUÇÃO



Principais diferenças entre Windows e Linux

01

Linux

Usabilidade: Possui algumas complexidades de instalação, mas conclui tarefas complexas mais facilmente.

Segurança: É considerado muito mais seguro; hackers acham difícil se aproveitar de suas brechas.

02

Windows

Usabilidade: Oferece um sistema simples para operar, mas pode levar mais tempo para ser totalmente instalado.

Segurança: É o principal alvo de malwares, tornando-se vulnerável sem um bom software antivírus ativo.

USABILIDADE E SEGURANÇA



Principais diferenças entre Windows e Linux

01

Linux

Suporte: O Linux tem suporte por meio de uma enorme comunidade de fóruns de usuários/sites e busca on-line.

Atualizações: No Linux, os usuários têm controle total das atualizações, podem instalá-las sempre que precisarem e elas levam menos tempo, sem qualquer reinicialização.

02

Windows

Suporte: O Windows tem um suporte que é facilmente acessível, fóruns on-line/sites e suporte pago também.

Atualizações: No Windows, as atualizações podem ocorrer a qualquer momento. Geralmente são obrigatórias e sem opções de cancelamento em momentos inconvenientes.

SUPORTE E ATUALIZAÇÕES



Funcionalidades de um S.O

01

Interface Gráfica (GUI)

Os usuários clicam em elementos visuais da tela (como ícones e botões) para interagir com o sistema de forma prática e intuitiva.

02

Linha de Comando (CLI)

Os usuários digitam comandos textuais diretamente no terminal para instruir o sistema operacional e executar tarefas.

INTERFACE USUÁRIO-HARDWARE



Funcionalidades de um S.O

03

Coordenar Componentes de Hardware

Comunicação Unificada: O sistema operacional coordena os componentes de hardware, traduzindo as instruções para que cada dispositivo funcione de forma integrada.

Drivers de Dispositivo: Utiliza softwares de tradução específicos (drivers) para permitir que o sistema converse com diferentes hardwares e softwares com sucesso.

COORDENAÇÃO E DRIVERS



Funcionalidades de um S.O

04

Gerenciamento de Dispositivos

Controle Centralizado: Permite que o sistema operacional monitore e gerencie diretamente todos os componentes de hardware conectados ao computador.

Drivers e Recursos: Facilita a atualização de drivers, bem como a ativação ou desativação de periféricos de forma rápida e segura.



GERENCIAMENTO DE DISPOSITIVOS



Funcionalidades de um S.O

05

Fornecer Ambiente de Software

Suporte de Execução: O sistema operacional fornece um ecossistema estruturado para que aplicativos e softwares funcionem perfeitamente.

Interface Consistente (GUI): Em sistemas como Windows e Mac OS, os aplicativos rodam em uma área de trabalho gráfica uniforme e intuitiva.



AMBIENTE DE SOFTWARE E GUI



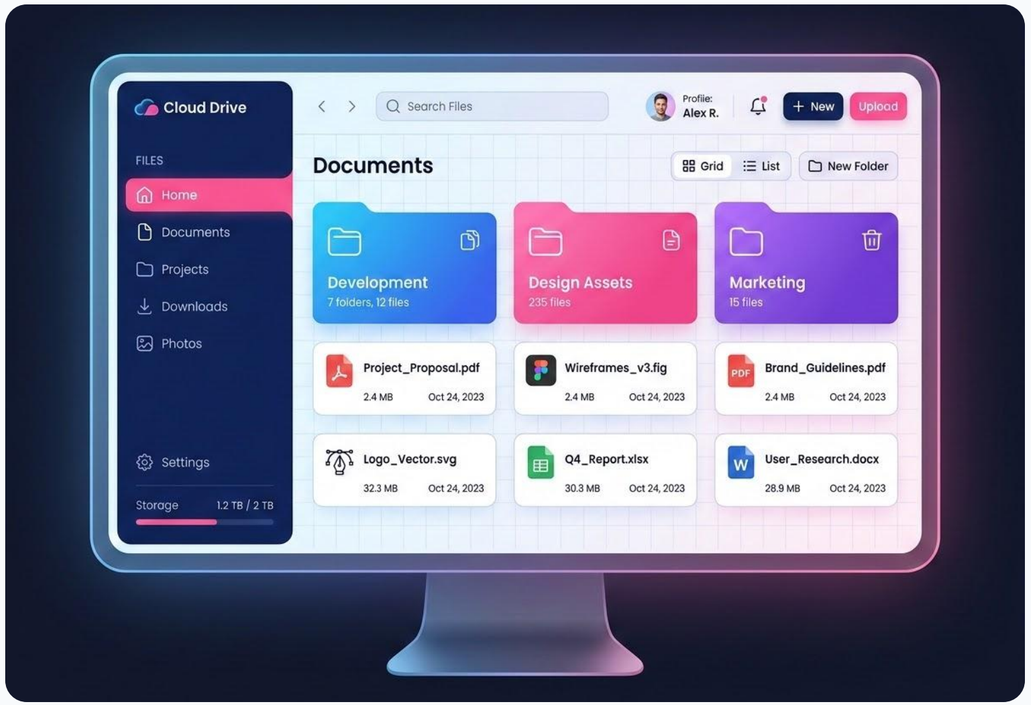
Funcionalidades de um S.O

03

Gerenciamento de Dados

Estrutura Organizada: O S.O. exibe uma estrutura de diretórios para o gerenciamento de dados de forma intuitiva.

Manipulação Total: Permite visualizar listagens de arquivos e pastas para mover, copiar, renomear e excluir dados facilmente.



ESTRUTURA E GERENCIAMENTO DE DADOS



Funcionalidades de um S.O

04

Monitoramento de Sistema

Desempenho de Hardware: Acompanha o uso de CPU, velocidade dos discos rígidos e o tráfego de rede para garantir eficiência.

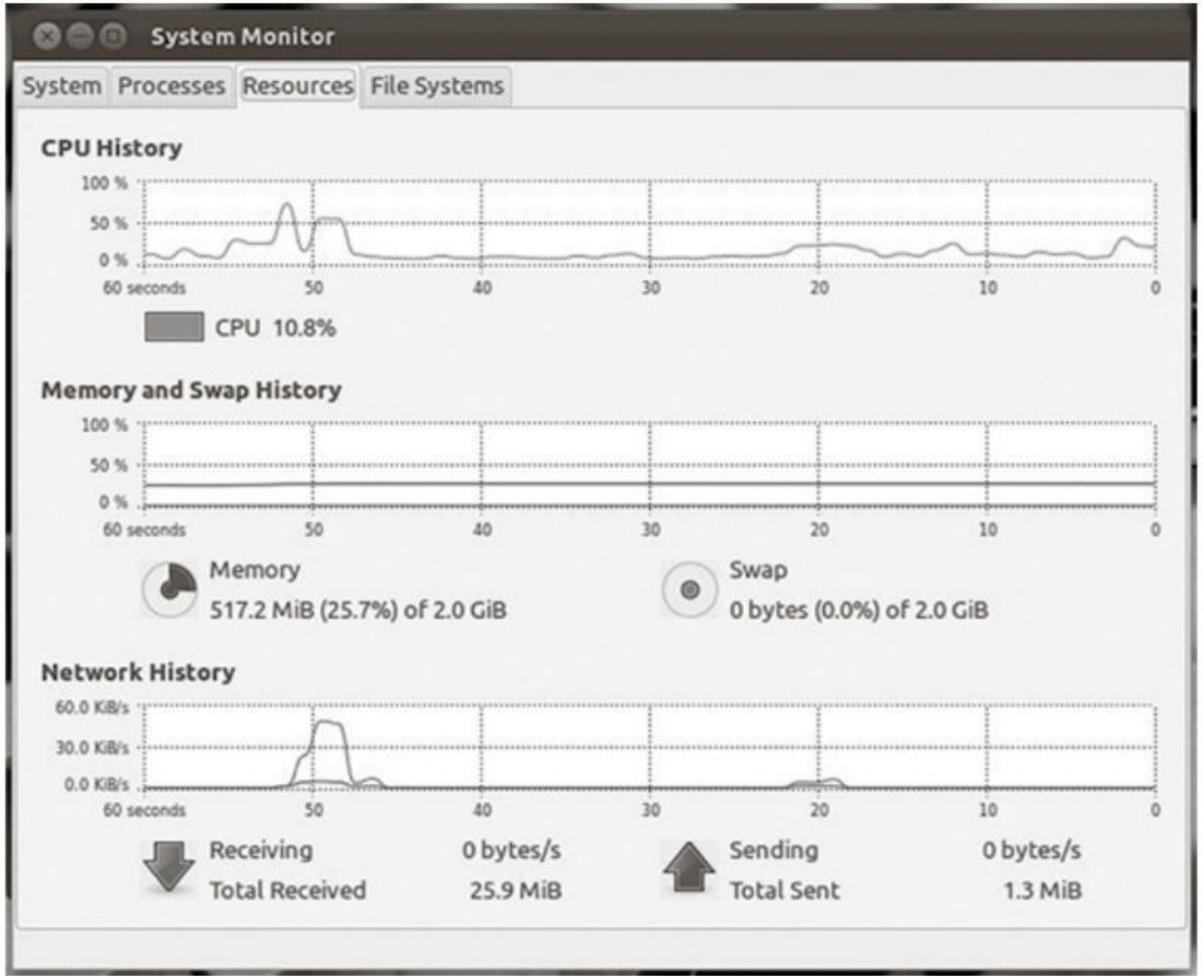
Segurança Ativa: Monitora continuamente as atividades do sistema em busca de malwares e ameaças cibernéticas.



MONITORAMENTO E INTEGRIDADE DO SISTEMA



Funcionalidades de um S.O



Parte II

Chamadas de Sistema

(System Calls)

Próximo Tópico:

- Programas de Sistema





Chamadas do Sistema

Conceito Chave

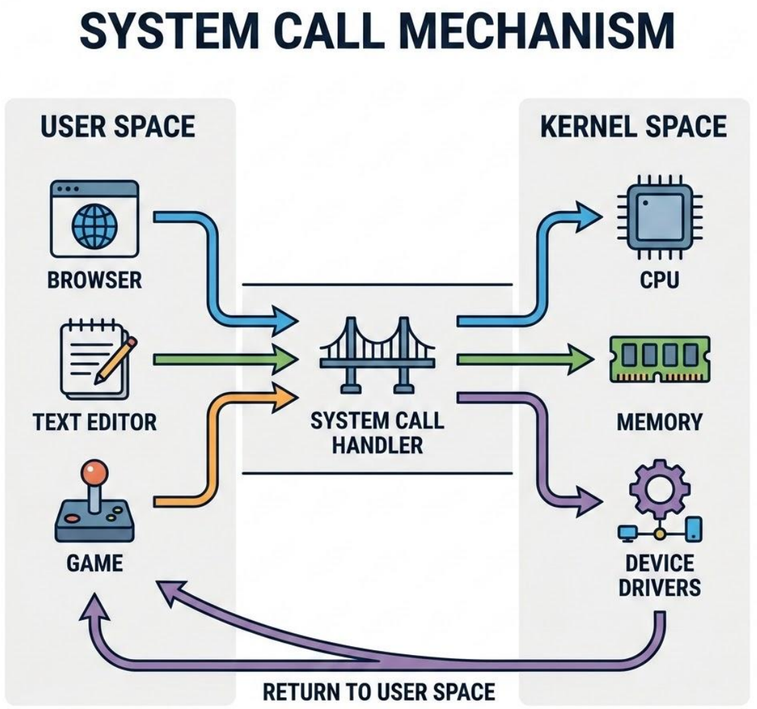
Uma chamada de sistema é uma rotina essencial que atua como ponte de transição.

Ela permite que um aplicativo de usuário solicite de forma segura ações e recursos que requerem privilégios especiais do hardware.

Funcionamento

São interfaces usadas pelas aplicações para solicitar serviços diretamente ao **kernel** do sistema operacional.

Estas instruções são executadas com maior nível de privilégio quando comparadas às instruções comuns do espaço do usuário.





Chamadas de Sistema

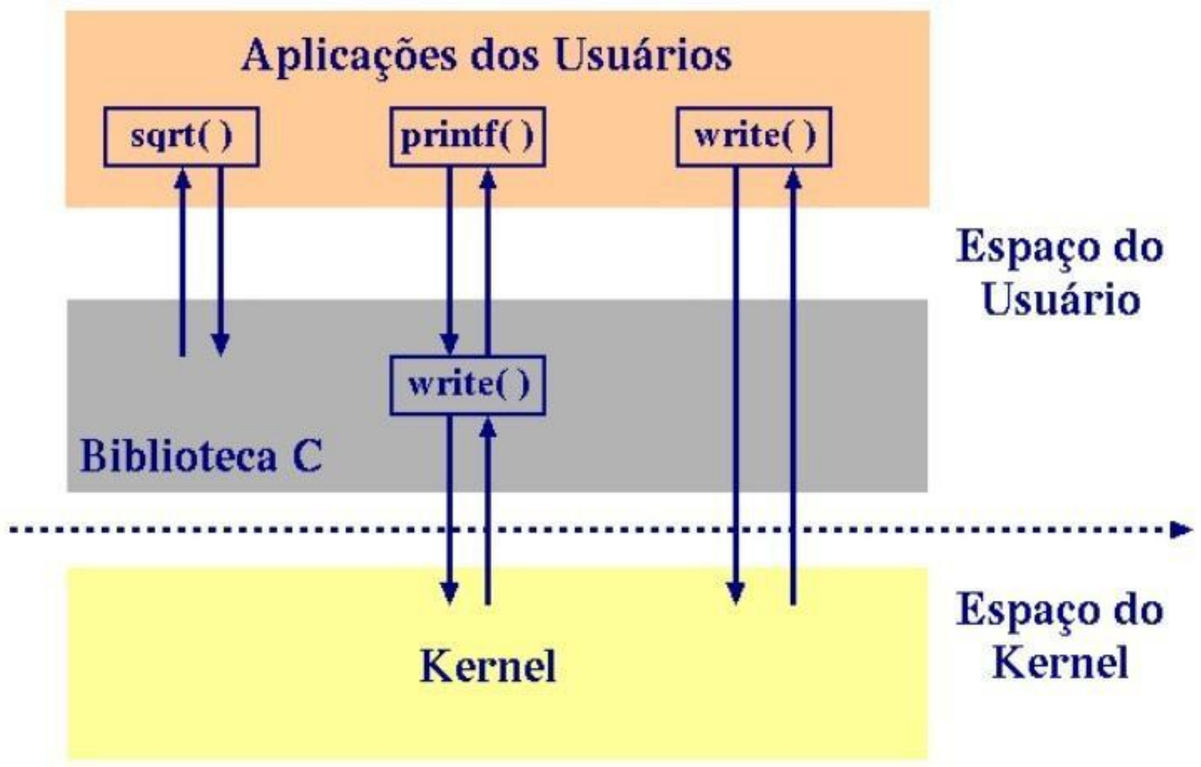
Linguagem C

Análise de Fluxo de Execução

No exemplo apresentado, a única chamada de sistema real é:

write()

- **sqrt()**: É executada totalmente no Espaço do Usuário (função matemática).
- **printf()**: É uma função de biblioteca que internamente faz a chamada de sistema **write()** para exibir a saída.





Chamadas do Sistema

Linguagem C

```
1  #include < stdio .h
2  >int main ( void )
3   { printf (" Hello World !\n")
4     ; return 0;
   }
5
```



Assembly

```
#------------------------------------------------------------------
# Writes "Hello World!" to the console using only system calls.
# Runs on 64-bit Linux only.
# To assemble and run:
# gcc -c hello2.s
# ld -o hello2 hello2.o
# ./hello2
#------------------------------------------------------------------
        .global _start

        .text
_start:
        # write(1, message, 13)
        mov $1, %rax                # system call 1 is write
        mov $1, %rdi                # file handle 1 is stdout
        mov $message, %rsi          # address of string to output
        mov $13, %rdx               # number of bytes
        syscall                     # invoke operating system to do write

        # exit(0)
        mov $60, %rax               # system call 60 is exit
        xor %rdi, %rdi              # we want return code 0
        syscall                     # invoke operating system to exit
message:
        .ascii "Hello World!\n"}
```



Chamadas do Sistema

Premissa Básica

É interessante ter o seguinte em mente: qualquer computador com apenas uma CPU pode executar apenas uma instrução por vez.

Transição de Controle

Se um processo estiver executando um programa no modo usuário e precisar de um serviço do sistema, ele terá de executar uma instrução de interrupção, ou de chamada de sistema, para transferir o controle para o sistema operacional.



Exemplo de Transição de Controle

1. Modo Usuário

Execução Comum

O programa roda no espaço do usuário com restrições de hardware.

```
printf("Hello World");
```

Instrução de Interrupção

→

2. Modo Kernel (Sistema Operacional)

Tratamento

O controle é transferido ao S.O., que inspeciona os parâmetros e executa a operação privilegiada no hardware.

```
# Chamada do S.O.
mov $1, %rax
mov $1, %rdi
syscall
```

O hardware executa a escrita física através de instruções privilegiadas do kernel.



Chamadas do Sistema - Gerenciamento de Processos

- Criação de processos
- Espera a finalização de processo
- Termina a execução do processo
- Informação sobre processos



Chamadas do Sistema - Gerenciamento de Processos

Funcionamento da Chamada fork()

1. O Mecanismo de Criação

Como o S.O. duplica o processo:

- Cria uma cópia idêntica do processo pai.
- Retorna **PID = 0** para o novo processo Filho.
- Retorna o **PID do Filho** para o processo Pai.

Exemplo Prático:

```
pid_t pid = fork();
```

2. Sincronização e Espera (wait)

Ciclo de Vida:

- **Processo A (Pai)** executa duas chamadas fork() consecutivas para gerar os filhos **B** e **C**.
- **Chamada wait C;** coloca o Pai em estado **Bloqueado** até que C conclua sua execução.
- **Chamada wait B;** sincroniza o término do processo B, liberando os recursos do sistema de forma segura.



Chamadas do Sistema - Sinais

1. Comunicação Inesperada

Situações Fora do Fluxo Padrão

Embora a maioria das formas de comunicação entre processos seja planejada, existem cenários críticos nos quais é necessária uma interrupção imediata e inesperada.

2. Exemplo Prático & Conexão

Interrompendo um Editor Travado

Se um usuário acidentalmente solicita a listagem de um arquivo gigante e quer parar a execução, o sistema precisa de um mecanismo para notificar e interromper o editor.

Como o S.O. resolve isso?

Através de **Sinais do Sistema** (como sigaction, kill, pause e alarm), detalhados na tabela a seguir.



Chamadas do Sistema - Sinais

01 Rotinas para execução de operações que interrompem a execução de um processo corrente

02 Informar exceções detectadas pelo hardware

03 Definição de limites de tempo

04 Suspender um processo por tempo indeterminado

05 Habilitar/desabilitar a recepção de sinais em um processo



Chamadas do Sistema

Sinais	s = **sigaction**(sig, &act, &oldact)	Define a ação a ser executada nos sinais
	s = **sigreturn**(&context)	Retorna de um tratamento de sinal
	s = **sigprocmask**(how, &set, &old)	Examina ou altera a máscara do sinal
	s = **sigpending**(set)	Obtém o conjunto de sinais bloqueados
	s = **sigsuspend**(sigmask)	Substitui a máscara do sinal e suspende o processo
	s = **kill**(pid, sig)	Envia um sinal para um processo
	residual = **alarm**(seconds)	Configura um temporizador
	s = **pause**()	Suspende o processo que fez a chamada até o próximo sinal



Chamadas do Sistema - Gerenciamento de Arquivos

01 Criação/abertura de arquivos

02 Fechamento de arquivos

03 Leitura/escrita de arquivos

04 Movimentação do ponteiro do arquivo



Chamadas do Sistema

Gerenciamento de arquivos	fd = **creat**(name, mode)	Modo obsoleto de criar um novo arquivo
	fd = **mknod**(name, mode, addr)	Cria um *i-node* normal, especial ou de diretório
	fd = **open**(file, how, ...)	Abre um arquivo para leitura, escrita ou ambos
	s = **close**(fd)	Fecha um arquivo aberto
	n = **read**(fd, buffer, nbytes)	Lê dados de um arquivo para um buffer
	n = **write**(fd, buffer, nbytes)	Escreve dados de um buffer em um arquivo
	pos = **lseek**(fd, offset, whence)	Move o ponteiro de arquivo
	s = **stat**(name, &buf)	Obtém informações de status de um arquivo
	s = **fstat**(fd, &buf)	Obtém informações de status de um arquivo
	fd = **dup**(fd)	Aloca um novo descritor de arquivo para um arquivo aberto
	s = **pipe**(&fd[0])	Cria um *pipe*
	s = **ioctl**(fd, request, argp)	Executa operações especiais em um arquivo
	s = **access**(name, amode)	Verifica a acessibilidade de um arquivo
	s = **rename**(old, new)	Atribui um novo nome a um arquivo
	s = **fcntl**(fd, cmd, ...)	Travamento de um arquivo e outras operações



Chamadas do Sistema - Gerenciamento de Diretórios

01 Criação/remoção de diretórios

02 Manipulação de links

03 (Des)Montagem no sistema de arquivos

04 Mudar diretório atual

05 Mudar diretório root



Chamadas do Sistema

Gerenciamento de diretórios e do sistema de arquivos	s = **mkdir**(name, mode)	Cria um novo diretório
	s = **rmdir**(name)	Remove um diretório vazio
	s = **link**(name1, name2)	Cria uma nova entrada, *name2*, apontando para *name1*
	s = **unlink**(name)	Remove uma entrada de diretório
	s = **mount**(special, name, *flag*)	Monta um sistema de arquivos
	s = **umount**(special)	Desmonta um sistema de arquivos
	s = **sync**()	Transfere todos os blocos da cache para o disco
	s = **chdir**(dirname)	Muda o diretório de trabalho
	s = **chroot**(dirname)	Muda o diretório-raiz



Chamadas do Sistema - Proteção

01 Modificar permissões de acesso em arquivos

02 Informações de usuário

03 Mudar o proprietário de um arquivo



Chamadas do Sistema

Proteção	s = **chmod**(name, mode)	Altera os bits de proteção de um arquivo
	uid = **getuid**()	Obtém a *uid* do processo que fez a chamada
	gid = **getgid**()	Obtém a *gid* do processo que fez a chamada
	s = **setuid**(uid)	Configura a *uid* do processo que fez a chamada
	s = **setgid**(gid)	Configura a *gid* do processo que fez a chamada
	s = **chown**(name, owner, group)	Altera o proprietário e o grupo de um arquivo
	oldmask = **umask**(complmode)	Altera a máscara de modo



Chamadas do Sistema - Gerência do Tempo

01 Verificar hora atual (segundos desde 1 de janeiro de 1970)

02 Modificar hora

03 Modificar os horários de acesso de um arquivo

04 Informações sobre uso de CPU pelo usuário (Tempo gasto pelo usuário nos modos de usuário e sistema)



Chamadas do Sistema

Gerenciamento de tempo	seconds = **time**(&seconds) s = **stime**(tp) s = **utime**(file, timep) s = **times**(buffer)	Obtém o tempo decorrido desde 1º de janeiro de 1970 Configura o tempo decorrido desde 1º de janeiro de 1970 Configura o momento do "último acesso" de um arquivo Obtém os tempos do usuário e do sistema usados até o momento

Parte III - Projeto e Iplementação de S.O



Objetivos do projeto de um S.O

O primeiro problema ao projetar um sistema é a definição de objetivos e especificações.

Em um nível mais alto, o projeto do sistema será afetado pela escolha do hardware e do tipo de sistema:

- **batch**
- **tempo compartilhado**
- **monousuário, multiusuário**
- **distribuído**
- **tempo real**
- **uso geral**



Objetivos do Projeto de um S.O.

Além de definir o hardware e modelo, a especificação detalhada de requisitos é complexa.
Essas metas de projeto são tradicionalmente divididas em dois grandes grupos:

Objetivos de Usuário

Definem o que o usuário final espera da experiência ao interagir com o sistema operacional.

- **Conveniência e facilidade de uso**
- **Fácil aprendizado**
- **Confiabilidade, segurança e velocidade**

Objetivos do Sistema

Definem os requisitos sob a ótica de quem projeta, implementa e mantém o software básico.

- **Fácil de projetar e manter**
- **Alta confiabilidade e robustez**
- **Eficiência no uso dos recursos**



Objetivos de usuário

O sistema deve ser:

- **conveniente para usar;**
- **fácil de aprender e utilizar;**
- **confiável;**
- **seguro;**
- **veloz.**

Nota importante

Naturalmente, essas especificações não são particularmente úteis ao projeto do sistema, já que não há um consenso geral sobre como alcançá-las.



Objetivos do sistema

O sistema deve ser:

- fácil de projetar, implementar e manter;
- confiável;
- sem erros;
- eficiente.

Nota importante

Novamente, esses requisitos são vagos e podem ser interpretados de várias maneiras.



Em resumo...

Não há uma solução única!

É impossível desenvolver um S.O que atenda a todos os requisitos que existem ou possam surgir.

VxWorks

STR para Embarcados

Focado em altíssima confiabilidade, tempo de resposta determinístico e suporte a recursos de hardware extremamente limitados.

≠

MVS

Sistema de Mainframe da IBM

Focado em processamento de dados em larga escala, máxima disponibilidade comercial, segurança robusta e altíssimo throughput de transações.



Mecanismos e Políticas

Primeiro, vamos separar os conceitos de política e mecanismo.

Mecanismo

Como fazer algo

Determina de que forma o sistema executa uma ação ou disponibiliza um recurso.

Política

O que deve ser feito

Determina quais decisões tomar ou quais regras de negócio devem ser aplicadas.

Exemplo Prático

O construtor **timer** é um **mecanismo** que assegura a proteção da CPU.

No entanto, a decisão de por quanto tempo ele deve ser posicionado, para um usuário específico, é uma **decisão política**.



Mecanismos e Políticas

Em um cenário ideal, um mecanismo deve ser insensível à política.

O Exemplo do Mecanismo

Considere um mecanismo projetado para dar prioridade a determinados tipos de programas sobre outros. Se ele for apropriadamente separado da política, pode ser usado de duas formas opostas:

Política A

Priorizar I/O-Intensivos

Configura o sistema para garantir que programas dependentes de Entrada/Saída tenham prioridade máxima sobre programas CPU-intensivos.

Política B (Oposta)

Priorizar CPU-Intensivos

Configura o mesmo mecanismo de forma invertida, dando prioridade para programas que dependem de processamento intensivo (CPU).



Implementação do S.O







Implementação do S.O



Quem desenvolve o IOS?





Implementação do S.O



Quem desenvolve o Android?

SAMSUNG

MOTOROLA



Implementação do S.O

Complexidade do Desenvolvimento

Como os sistemas operacionais são formados por vastos conjuntos de programas, desenvolvidos por milhares de colaboradores ao longo de décadas, torna-se inviável formular uma única definição sobre sua estrutura geral de implementação.

Linguagem C & Assembly

Os núcleos (kernels) de sistemas amplamente difundidos como Linux e Windows são essencialmente escritos em C. Pequenos trechos em linguagem de montagem são reservados para controle de drivers e estados físicos de registradores.



Implementação do S.O

Linguagens de Alto Nível

Atualmente, embora alguns sistemas operacionais ainda sejam escritos em linguagem de montagem, a maioria é escrita em uma linguagem de mais alto nível como C ou de nível ainda mais alto como C++.

Múltiplas Linguagens

O desenvolvimento de um sistema operacional moderno não se limita a um único código. Ele também pode ser escrito em mais de uma linguagem de programação, combinando diferentes tecnologias para obter a melhor performance.



Implementação do S.O

Vantagens de se programar S.Os em linguagem de alto nível:

Desenvolvimento Rápido

• **Escrita ágil:** Programas são desenvolvidos muito mais rapidamente do que em Assembly.• **Fácil de entender:** O código possui maior clareza e legibilidade.• **Fácil de depurar:** A localização e correção de erros são simplificadas.

Eficiência & Portabilidade

• **Mais compacto:** Expressa algoritmos complexos com menos linhas de código.• **Portabilidade:** O sistema operacional pode ser compilado e executado em diferentes arquiteturas de hardware.



Implementação do S.O

Vantagens de se programar S.Os em linguagem de alto nível: Exemplos

MS-DOS

Baixa Portabilidade: Desenvolvido diretamente em linguagem de máquina Intel 8088. Por conta disso, só era capaz de operar em processadores da arquitetura Intel x86.

Linux

Alta Portabilidade: Escrito praticamente todo na linguagem de alto nível C. Isso permite que o sistema operacional seja facilmente compilado e opere em uma enorme quantidade e variedade de processadores.



Implementação do S.O

Por outro lado, as únicas desvantagens são:

Diminuição da Velocidade

Menor eficiência: O código gerado por compiladores de alto nível pode apresentar overhead, resultando em menor velocidade comparado à otimização manual em Assembly.

Aumento de Requisitos

Maior consumo: A complexidade e as estruturas de abstração exigem maior quantidade de memória e recursos de processamento do hardware.

Parte III - Estrutura do S.O

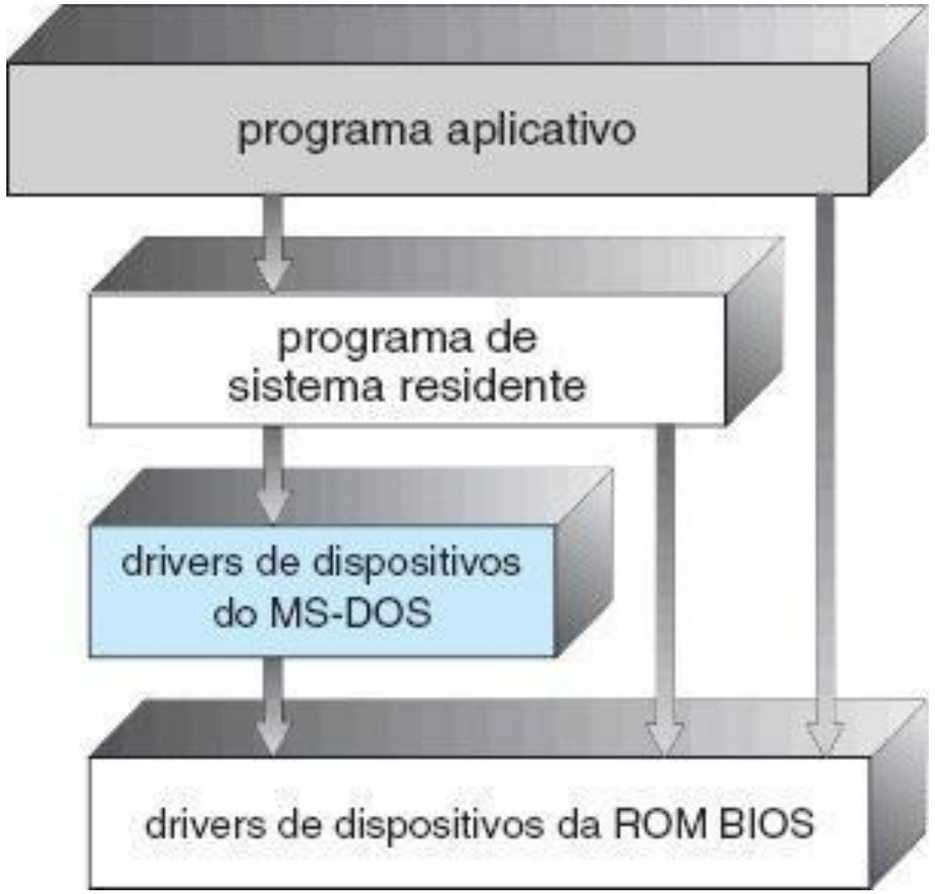


Estrutura do S.O

MS-DOS

Ausência de Barreiras: Muitos sistemas operacionais não têm estruturas bem definidas.

No MS-DOS, os programas aplicativos podiam acessar diretamente as rotinas básicas do sistema, comprometendo a estabilidade e a segurança.





Estrutura do S.O

UNIX

Estrutura Monolítica: Muitos sistemas operacionais não têm estruturas bem definidas.

- Componentes do kernel não possuem barreiras!



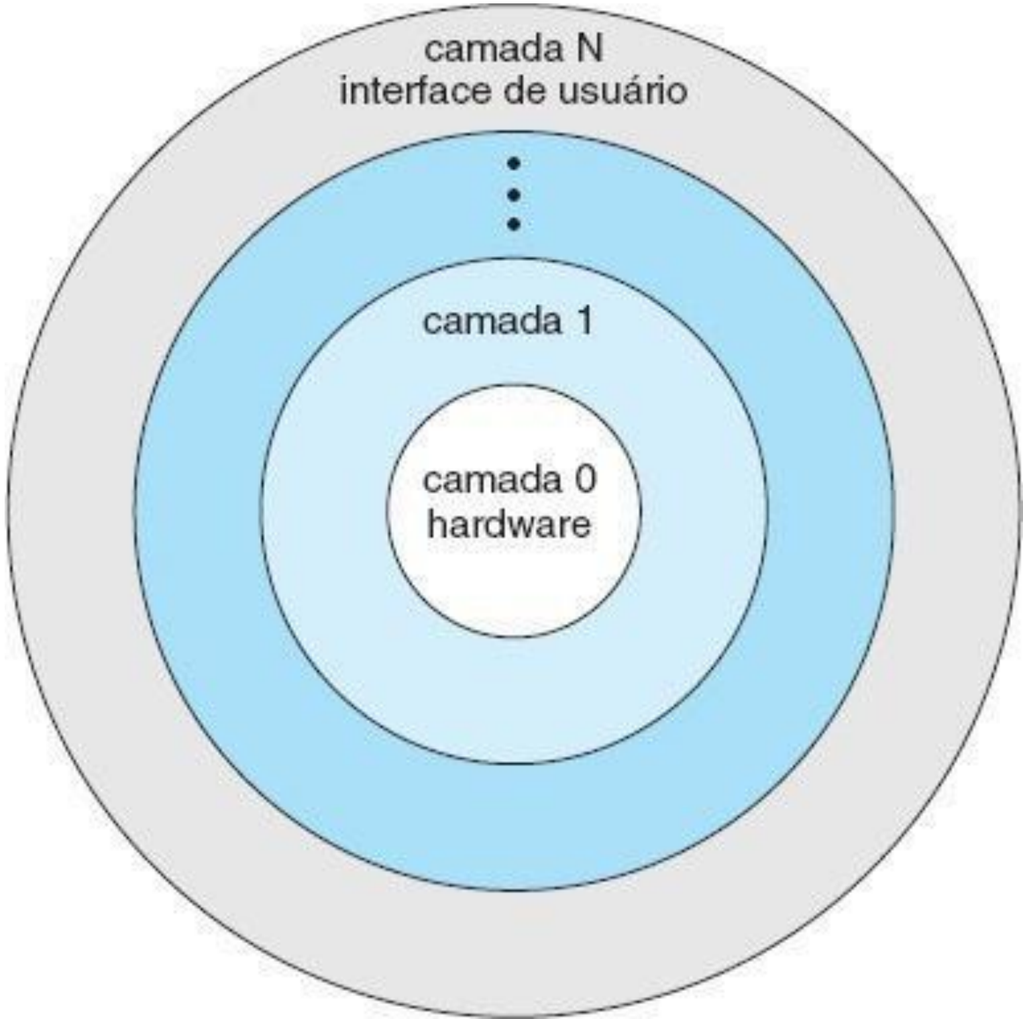


Abordagem em camadas

Conceito Principal

O sistema operacional é estruturado e dividido em várias camadas ou níveis:

- **Camada Inferior (Camada 0):** O hardware propriamente dito.
- **Camada Superior (Camada N):** A interface de usuário para interação.



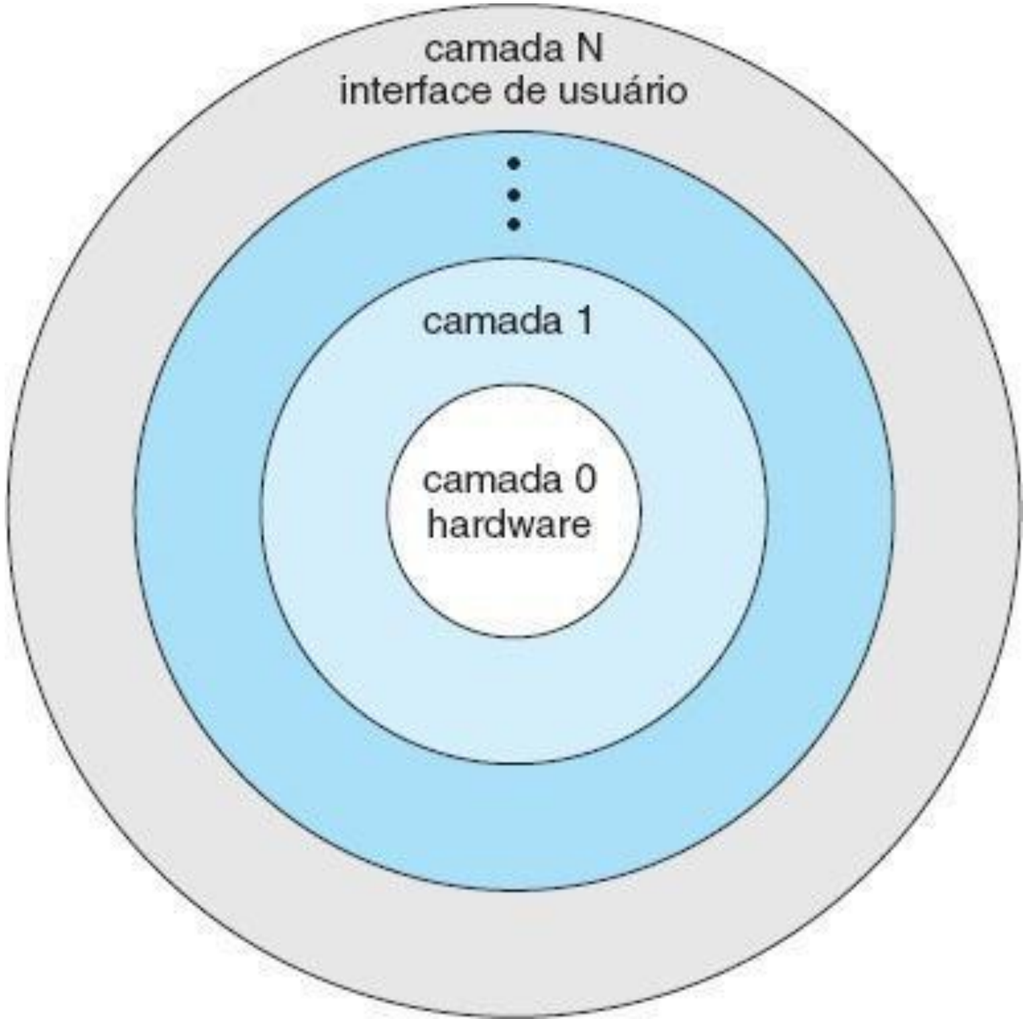


Abordagem em camadas

Interação entre Níveis

Uma camada típica de sistema operacional (**camada M**) é composta por estruturas de dados e rotinas específicas.

- **Invocação ascendente:** Suas rotinas podem ser invocadas por camadas de níveis mais altos.
- **Invocação descendente:** A camada M pode, por sua vez, invocar operações de níveis mais baixos.



Abordagem em camadas

Exemplo Prático de Interação

Consideremos uma operação comum: ler dados de um arquivo no disco. Essa requisição flui de forma ordenada através das camadas do sistema operacional.

- **Invocação Descendente (Requisição):** A interface de usuário (camada superior) solicita um arquivo. Essa requisição passa pela camada de gerenciamento de arquivos até chegar aos drivers de dispositivo (camadas inferiores) para ler o hardware físico.
- **Invocação Ascendente (Retorno):** O hardware realiza a leitura dos dados e envia o resultado de volta para cima, subindo nível a nível até ser finalmente exibido na tela para o usuário.



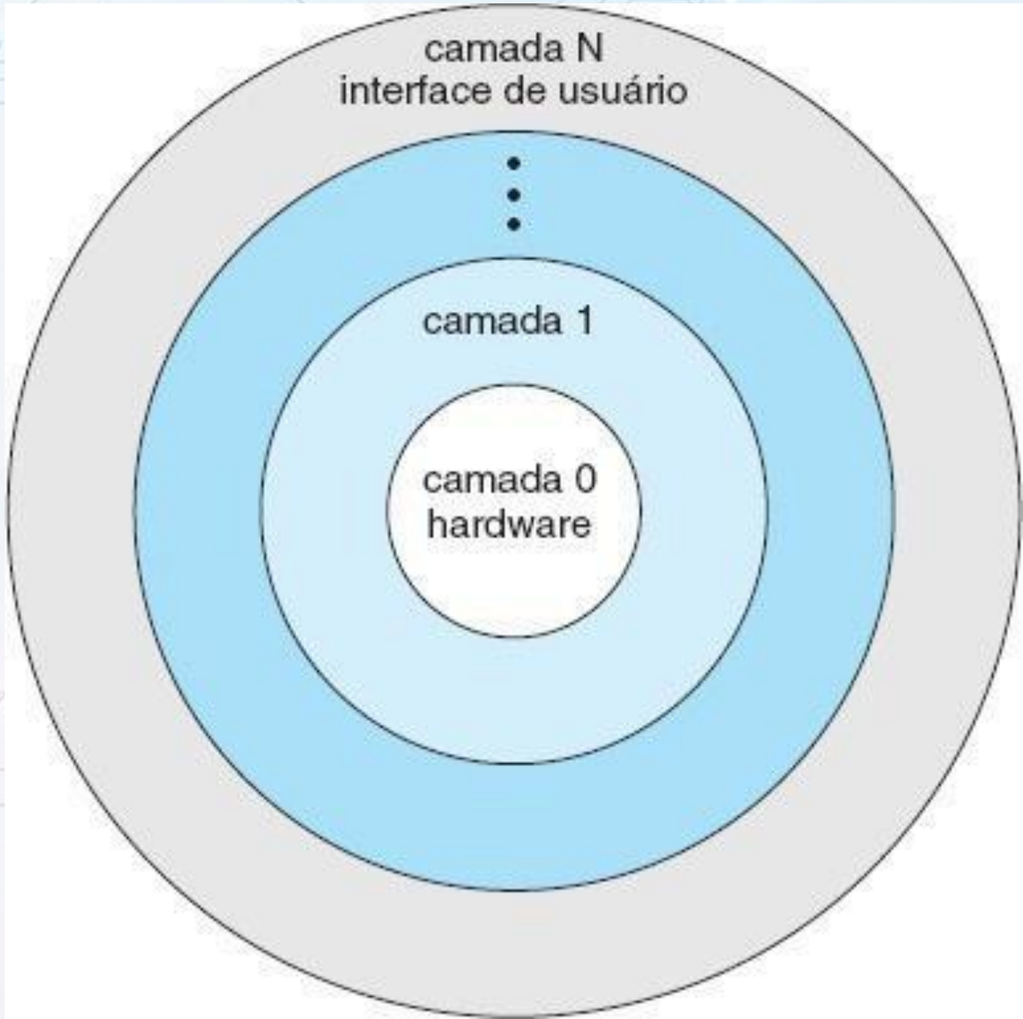
Abordagem em camadas

Vantagens

Simplicidade de construção e depuração.

Desvantagens

Já que uma camada pode usar somente camadas de nível mais baixo, é necessário um planejamento cuidadoso.





Microkernels

Arquitetura do Sistema

Esse método estrutura o sistema operacional removendo todos os componentes não essenciais do kernel e implementando-os como programas de nível de sistema e de usuário.

Comunicação & Processos

Normalmente, os microkernels fornecem um **gerenciamento mínimo dos processos** e da **memória**, além de um recurso de **comunicação**.

Exemplo: *se o programa cliente deseja acessar um arquivo, ele deve interagir com o servidor de arquivos. O programa cliente e o serviço nunca interagem diretamente.*



Microkernels

Isolamento de Serviços

Diferente de um SO monolítico, no microkernel o **Servidor de Arquivos** é um programa independente, executado fora do kernel no espaço do usuário.

A Mediação do Kernel

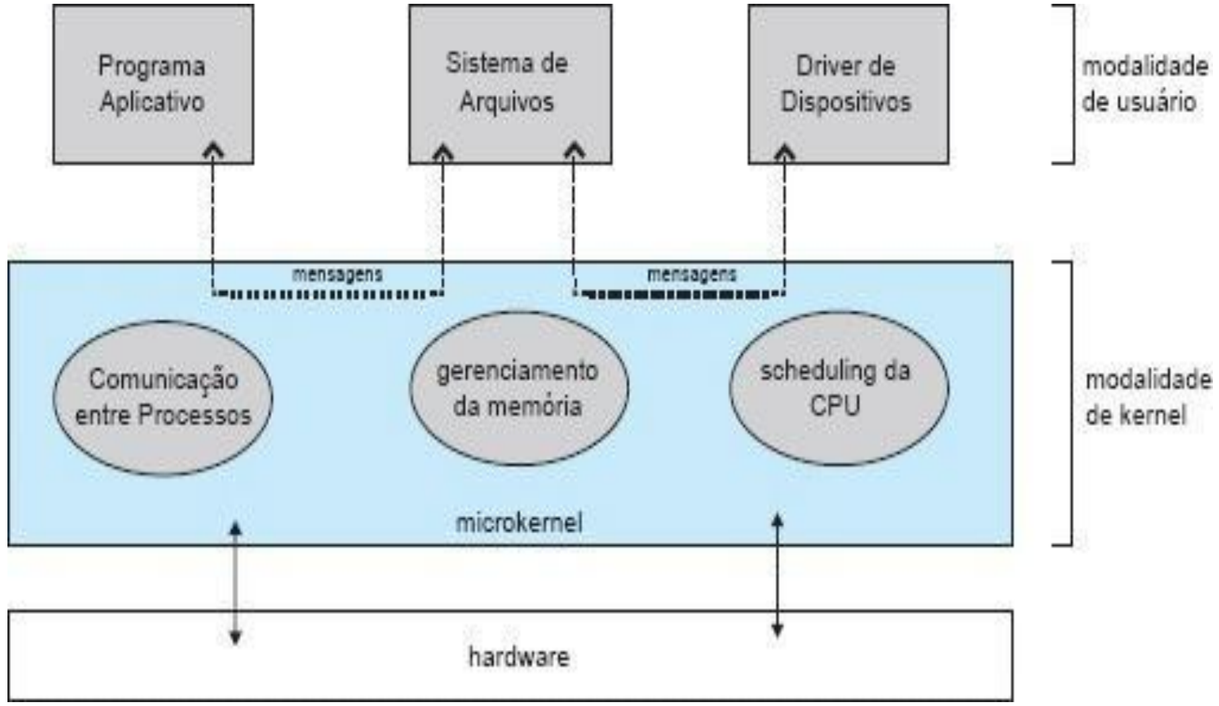
Quando um aplicativo precisa ler ou gravar arquivos, ele não acessa o disco diretamente; ele deve enviar uma mensagem de solicitação para o Microkernel.

Papel de Intermediário

O Microkernel atua como um hub: recebe a solicitação do cliente e a encaminha ao Servidor de Arquivos, que possui a lógica da operação.

Independência & Estabilidade

Cliente e serviço nunca interagem diretamente. Se o Servidor de Arquivos falhar, o kernel permanece intacto e o sistema continua operando de forma estável.





Microkernels

Vantagens do Modelo

Extensibilidade

Facilidade na extensão do S.O. Todos os serviços novos são adicionados ao **espaço do usuário** e, consequentemente, não requerem a modificação do kernel.

Maior Segurança

A maior parte dos serviços é executada em **modo usuário**, limitando privilégios e protegendo o núcleo do sistema contra possíveis falhas de segurança.

Tolerância a Falhas

Se um serviço falha, o resto do S.O. permanece **"de pé"**. Evita quedas generalizadas no sistema causadas por erros em drivers ou módulos externos.

Microkernels

Desvantagens do Modelo



Overhead de Desempenho

O principal ponto fraco deste modelo é o impacto na performance à medida que o sistema tende a expandir.

A comunicação frequente entre processos (IPC) e a constante alternância de contexto no espaço de usuário adicionam um atraso significativo em comparação aos sistemas monolíticos.



Módulos

Talvez a melhor abordagem atual para o projeto de S.O.

Vinculação Dinâmica

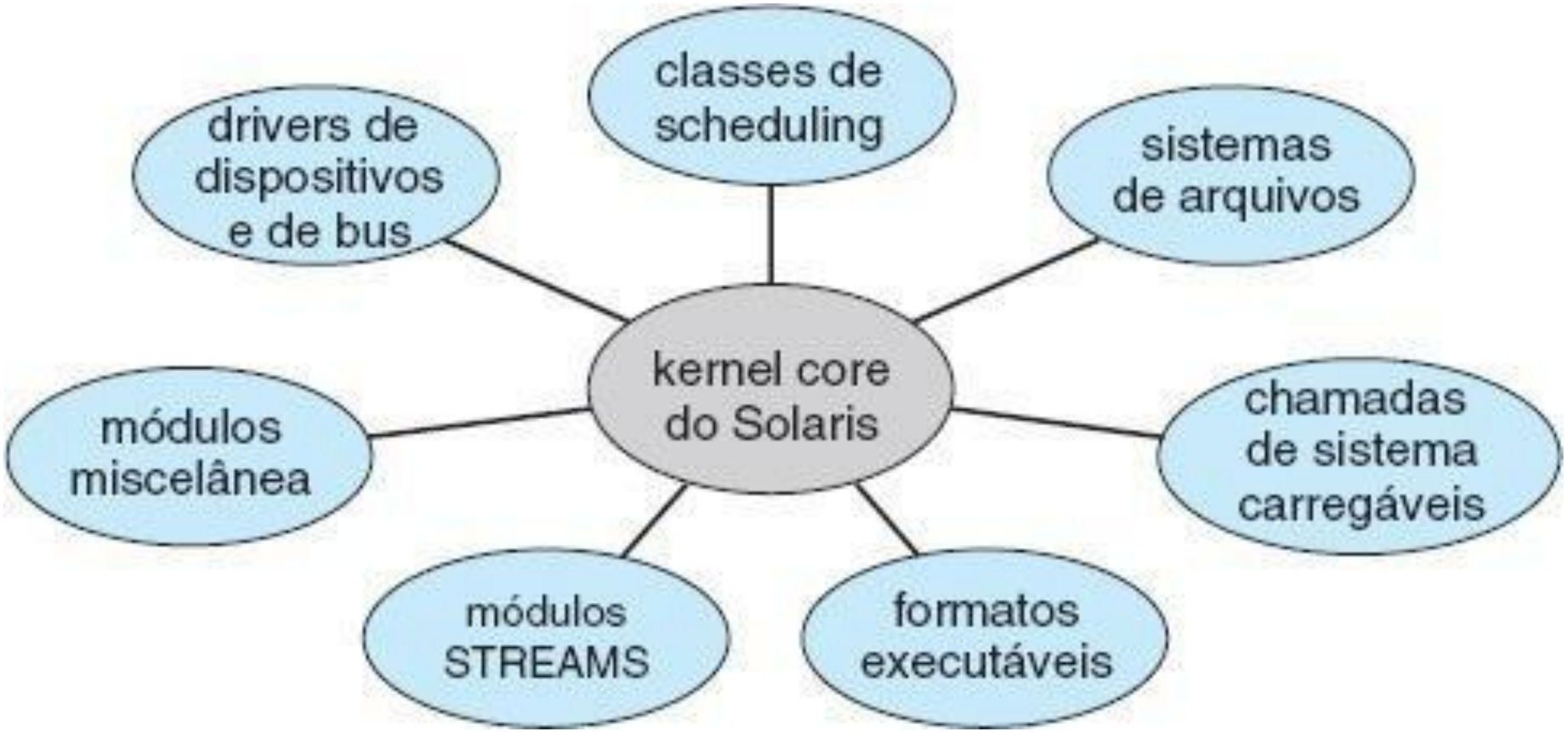
O kernel tem um conjunto de componentes nucleares e vincula serviços adicionais por meio de módulos, **tanto em tempo de inicialização quanto em tempo de execução**.

Sistemas Modernos

Esse tipo de implementação altamente modular é extremamente comum nos sistemas operacionais de hoje, incluindo **Unix, Solaris, Linux, Mac OS X e Windows**.



Módulos


classes de scheduling
drivers de dispositivos e de bus
sistemas de arquivos
kernel core do Solaris
módulos miscelânea
chamadas de sistema carregáveis
módulos STREAMS
formatos executáveis



Módulos

Relação com Camadas

Tem semelhanças com o modelo de camadas, mas **não necessita "pular"** de camada em camada para encontrar um serviço.

Relação com Microkernel

Também tem semelhanças com microkernel, porém, **é mais eficiente** pois os módulos não precisam invocar o serviço de transmissão de mensagens.

Comunicação Direta na Arquitetura Modular

Por que a arquitetura de módulos dispensa a troca de mensagens do Microkernel

Abordagem do Microkernel

No **Microkernel**, os serviços (como sistemas de arquivos e drivers) rodam no espaço do usuário. Para que um programa acesse um arquivo, ele precisa enviar uma mensagem formal via **IPC (Inter-Process Communication)**.

Esse processo envolve **troca de contexto** constante e cópias de dados entre o espaço de usuário e o kernel, o que adiciona uma latência significativa ao sistema operacional.

Abordagem Modular (LKM)

Na arquitetura de **Módulos Carregáveis (LKM)**, os módulos são carregados diretamente no espaço de endereçamento do kernel.

Como o subsistema de arquivos e o driver residem no mesmo espaço de memória do kernel core, um serviço pode chamar o outro **diretamente por meio de funções (chamadas de API diretas)**, eliminando completamente a necessidade de empacotar, transmitir e desempacotar mensagens de rede interna.



Sistemas Híbridos

Estruturas Rígidas

Na prática, muito poucos sistemas operacionais adotam uma **estrutura única rigidamente definida**.

Abordagem Híbrida

Sistemas Operacionais modernos combinam diferentes estruturas, resultando em **sistemas híbridos** que resolvem problemas de desempenho, segurança e usabilidade.



Sistemas Híbridos

Desempenho Monolítico

Linux e Solaris são monolíticos porque o desempenho é muito mais eficiente quando o sistema operacional ocupa um único espaço de endereçamento.

Flexibilidade Modular

No entanto, eles também são **modulares** para que novas funcionalidades possam ser adicionadas ao kernel dinamicamente.

O que é um Sistema Monolítico?

Uma arquitetura de sistema operacional em que os componentes do kernel (como gerenciamento de arquivos, drivers e escalonamento) operam sem barreiras internas rígidas ou uma segmentação bem definida.





Arquitetura Híbrida do Mac OS X

Estrutura do Kernel XNU

O Mac OS X utiliza o kernel **XNU**, um sistema híbrido que combina as vantagens de desempenho das arquiteturas **monolíticas** com a **modularidade** e segurança dos **microkernels**.

- **Parte Microkernel (Mach):** Gerencia as tarefas mais fundamentais do sistema, como threads, proteção de memória e IPC (comunicação entre processos).
- **Parte Monolítica (BSD-UNIX):** Fornece APIs estáveis do padrão POSIX, pilhas de rede, suporte a sistemas de arquivos e o modelo de processos tradicional.
- **Parte Modular (I/O Kit & Kexts):** Permite o carregamento dinâmico de drivers e extensões do kernel de forma modular, sem necessidade de recompilar todo o sistema.

Interface Gráfica (Aqua)
Espaço do Usuário / Camada Híbrida de Apresentação

Ambientes de Aplicação & APIs
Cocoa (Objective-C) | Java | Quicktime | BSD Env

Microkernel Mach
Gerenciamento de Threads, Memória Virtual, IPC e Mensagens básicos.

Subsistema BSD
Arquitetura Monolítica: Rede, Processos, Syscalls e Filesystems integrados.

I/O Kit & Extensões de Kernel (Modular)
Suporte a carregamento dinâmico de drivers orientados a objetos.



Arquitetura Híbrida do iOS

Base XNU (Darwin)

O iOS é construído sobre o **Darwin**, o mesmo núcleo híbrido do macOS.

Combina a flexibilidade do microkernel **Mach** com a robustez e rede do subsistema **BSD-UNIX**.

Otimização Móvel

Embora compartilhe a base, a camada superior é o **Cocoa Touch**.

Desenvolvido especificamente para telas multi-touch, sensores de movimento e gerenciamento rígido de energia.

Segurança & APIs

Não executa apps de macOS devido à segurança por **sandboxing** rígido.

As APIs exclusivas e as restrições de hardware garantem que os sistemas permaneçam isolados e protegidos.

Arquitetura do Android

Pilha em Camadas

O Android é estruturado como uma **pilha de software em camadas**, oferecendo um rico conjunto de frameworks para desenvolvimento móvel.

Esta arquitetura modular facilita a separação de responsabilidades, desde o hardware até os aplicativos finais de usuário.

Kernel Linux

A base do sistema utiliza o **Kernel Linux** para suporte de baixo nível.

- **Hardware:** Gerenciamento de memória e processos.
- **Drivers:** Abstração de dispositivos físicos.
- **Energia:** Gerenciamento otimizado para dispositivos móveis.

Tempo de Execução

O **Android Runtime** gerencia a execução eficiente das aplicações.

- **Bibliotecas:** Um conjunto de bibliotecas de núcleo fundamentais.
- **Dalvik / ART:** Máquina virtual otimizada para baixo consumo de memória e processamento.

Parte IV - Depuração do S.O



O que é a Depuração de um S.O.?

Definição Ampla

Em sentido amplo, depuração é a atividade de encontrar e corrigir erros em um sistema, tanto em hardware quanto em software.

Ajuste de Desempenho

Problemas de desempenho são considerados bugs e, portanto, a depuração também pode incluir o ajuste de desempenho que tenta melhorar o processamento removendo gargalos.



Análise de Falhas

Geração de Logs

Quando um processo falha, a maioria dos sistemas operacionais grava as informações de erro em um arquivo de log para alertar que um problema ocorreu.

Despejo de Núcleo

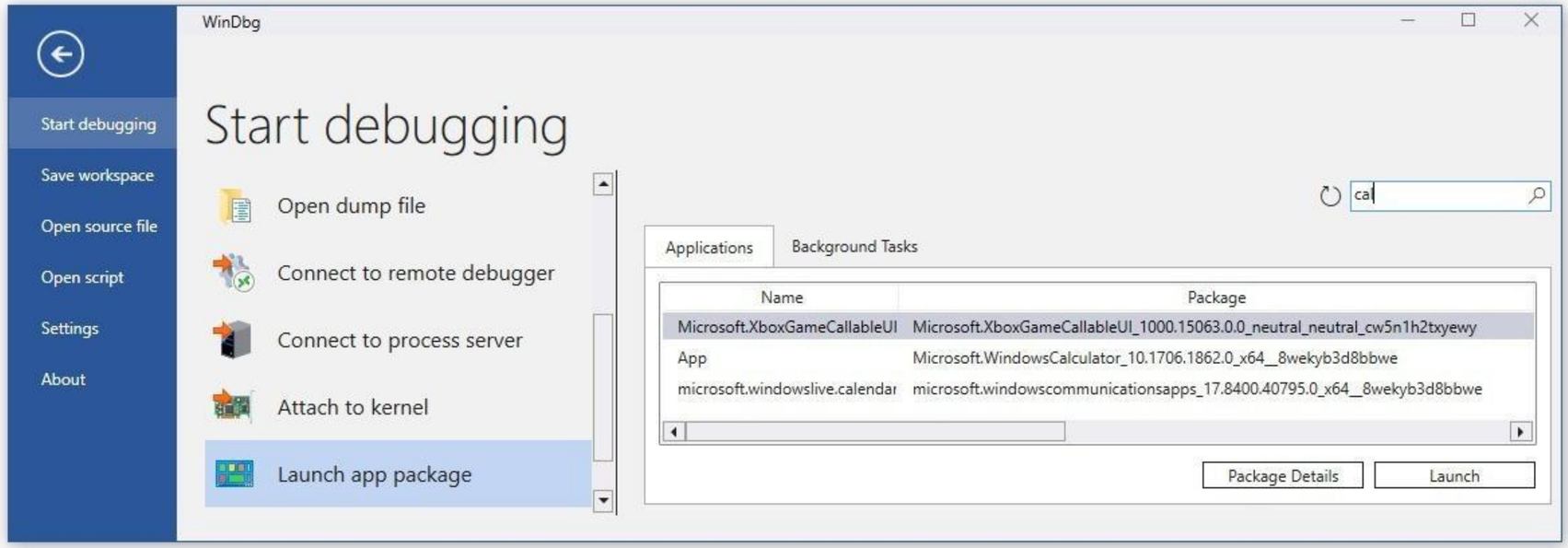
O sistema operacional também pode obter um despejo do núcleo (captura da memória do processo) e armazená-lo em um arquivo para análise posterior.

Análise de Falhas

Depuração de Processos

Programas em execução e despejos do núcleo podem ser examinados por um depurador, o que permite ao programador examinar o código e a memória de um processo.

WinDbg Preview



Análise de Falhas

O Desastre do Kernel

Uma falha no kernel é chamada de desastre. Quando ocorre um desastre, as informações de erro são salvas em um arquivo de log, e o estado da memória é salvo em um despejo de desastre.

Risco de Salvamento

Considere que uma falha de kernel no código do sistema de arquivos tornaria arriscado para o kernel tentar salvar seu estado em um arquivo do sistema de arquivos antes da reinicialização.

Análise de Falhas

Área de Disco Reservada

Uma técnica comum é salvar o estado da memória do kernel em uma seção de disco sem sistema de arquivos e reservada para essa finalidade.

Gravação do Kernel

Quando o kernel detecta um erro irrecuperável, ele grava todo o conteúdo da memória ou, pelo menos, as partes da memória do sistema que lhe pertencem, nessa área de disco.

Análise de Falhas



Coleta pós-Reinicialização

Quando o sistema reinicializa, um processo é executado para coletar os dados dessa área e gravá-los em um arquivo de despejo de desastre dentro de um sistema de arquivos, para análise.



Nível de Usuário

Obviamente, tais estratégias seriam desnecessárias para a depuração de processos em nível de usuário comum.

Análise de Falhas

LEI DE KERNIGHAN

“Depurar é duas vezes mais difícil do que escrever o código pela primeira vez. Portanto, se você escrever o código o mais inteligentemente possível, você não é, por definição, suficientemente inteligente para depurá-lo.”



Ajuste de Desempenho

Monitoramento

Para identificar gargalos, devemos ser capazes de monitorar o desempenho do sistema.

Métricas

Assim, o sistema operacional deve ter algum meio de calcular e exibir medidas de comportamento do sistema.

Rastreamento

Em vários sistemas, o sistema operacional faz isso produzindo listagens de rastreamento do comportamento do sistema.



Ajuste de Desempenho

A listagem de rastreamento funciona da seguinte forma:

1. Registro

Todos os eventos de interesse são registrados em log, com sua duração e parâmetros importantes, e gravados em um arquivo.

2. Análise

Posteriormente, um programa de análise pode processar o arquivo de log para determinar o desempenho do sistema e identificar gargalos e ineficiências.

3. Diagnóstico

Os rastreamentos também podem ajudar as pessoas a encontrar erros no comportamento do sistema operacional.



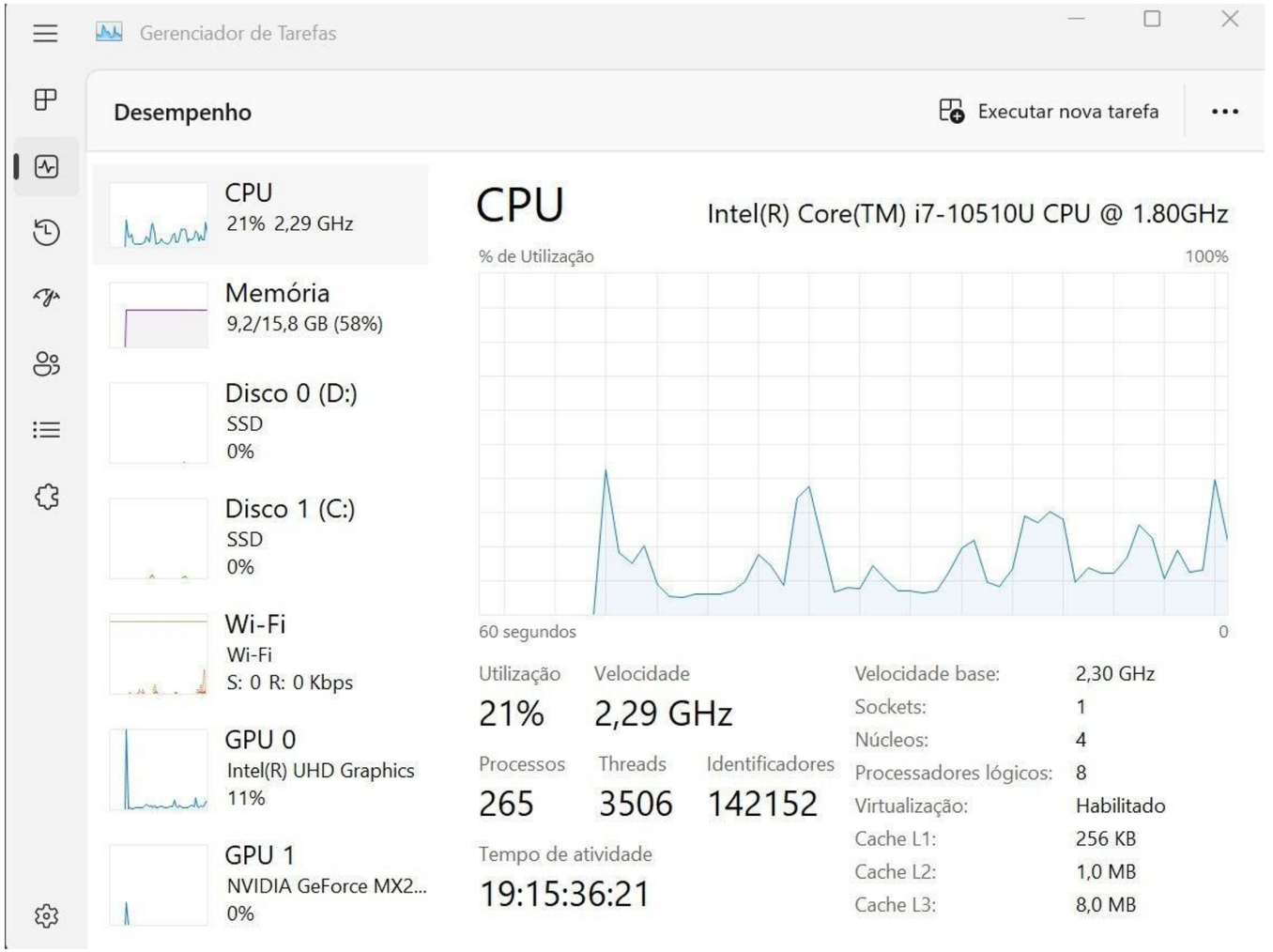
Ajuste de Desempenho

Ferramentas de Monitoramento do Sistema Operacional

Gerenciador de Tarefas do Windows

O Gerenciador de Tarefas do Windows é uma ferramenta essencial para monitoramento e análise em tempo real de sistemas Windows.

Ele inclui informações detalhadas sobre aplicativos e processos em execução corrente, uso de CPU e memória, além de estatísticas cruciais de rede e disco.



Parte V - Inicialização do S.O



Inicialização do S.O

O Desafio

Como o hardware sabe onde está o kernel ou como carregar esse kernel?

A Solução: Bootstrap

Na maioria dos sistemas de computação, um pequeno bloco de código conhecido como **programa bootstrap** ou **carregador bootstrap** localiza o kernel, carrega-o na memória principal e inicia sua execução.



Inicialização do S.O

Fase de Inicialização

• **Reinicialização da CPU:** Quando a CPU recebe um evento de reinicialização, o registrador de instruções é carregado com uma locação de memória predefinida.

• **Bootstrap Inicial:** A execução começa a partir dessa locação específica, onde se encontra o programa bootstrap inicial.

O Papel da ROM

• **Estado da RAM:** O programa fica na ROM (memória somente de leitura) porque a RAM está em um estado desconhecido durante a inicialização do sistema.

• **Segurança e Praticidade:** A ROM é conveniente pois não precisa ser inicializada e possui maior proteção contra infecções por vírus.



O programa Bootstrap

Múltiplas Tarefas

O programa bootstrap possui papel multifuncional, sendo capaz de executar diversas tarefas cruciais no momento em que o hardware é ligado.

Fluxo de Diagnóstico

- **Execução de Diagnósticos:** Geralmente, uma das primeiras tarefas é testar e determinar o estado físico e lógico da máquina.
- **Inicialização Segura:** Se os diagnósticos forem bem-sucedidos, o programa prossegue com as etapas de carregamento do sistema operacional.



Em dispositivos móveis...

Sistemas em ROM

Alguns sistemas armazenam o sistema operacional inteiro em ROM.

Cenários Adequados

O armazenamento do sistema operacional em ROM é adequado para sistemas operacionais pequenos, hardware de suporte simples e operação irregular.



Em dispositivos móveis...

Limitação da ROM

Um problema dessa abordagem é que a alteração do código bootstrap requer a mudança física dos chips de hardware da ROM.

Solução com EPROM

Alguns sistemas resolvem esse problema usando memória somente de leitura apagável e programável (EPROM), que funciona apenas para leitura, exceto ao receber um comando para se tornar gravável.



Observação

Hardware
Parte Física

Firmware (ROM)
Características híbridas

Software
Parte Lógica

Todos os tipos de ROM também são conhecidos como firmware, já que suas características se encaixam em algum ponto entre as características de hardware e as de software.



Limitações de Firmware

Desempenho Lento

Um problema comum com o firmware é que a execução do código nesse local é mais lenta do que em RAM.

Cópia para RAM

Alguns sistemas armazenam o sistema operacional em firmware e o copiam em RAM para execução rápida.



Em S.Os "grandes"... (Windows, Linux, Mac OS X)

Local de Armazenamento

Nesses sistemas, o carregador bootstrap é armazenado de forma segura em firmware (ROM), enquanto o sistema operacional principal permanece guardado no disco.

Execução do Bootstrap

O programa executa diagnósticos iniciais e lê um bloco específico em uma locação fixa do disco, enviando-o à memória para iniciar a inicialização do sistema.



Em S.Os "grandes"... (Windows, Linux, Mac OS X)

Bloco de Inicialização

O programa armazenado no bloco de inicialização pode ser suficientemente sofisticado para carregar o sistema operacional inteiro na memória e começar sua execução.

Busca do Kernel

Agora que o programa bootstrap inteiro foi carregado, ele pode percorrer o sistema de arquivos para encontrar o kernel do sistema operacional, carregá-lo na memória e iniciar sua execução.

Sistema Ativo

É somente nesse ponto, com todos os componentes essenciais devidamente carregados em memória, que o sistema operacional é considerado em execução.



Dúvidas?